Event Handling

Event handling is the mechanism used to respond to user actions in a Java program. Event handling means writing code to perform an action when the user interacts with a GUI component.

For example:

- Clicking a Button
- Typing in a TextField
- Moving or clicking the Mouse
- Pressing a Keyboard key
- Closing a Window

Classification of Events

Foreground Events	Background Events
Require **user interaction**.	Occur **without direct user interaction**.
Generated when the user performs an action.	Generated by the **OS or system**.
Examples: mouse click, key press, button click.	Examples: OS interrupts, operation completion, timer events.
User is directly involved.	User need not be directly involved.

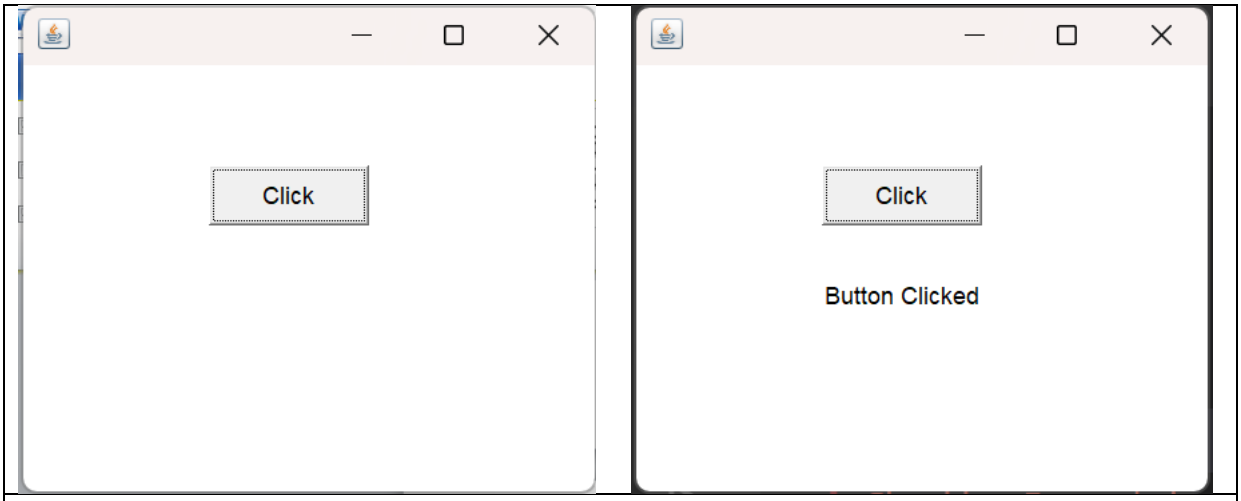
Event Class	Listener	Main Method(s)	Example / When it occurs
ActionEvent	ActionListener	actionPerformed()	Button clicked, menu item selected, Enter in TextField
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged()	Scrollbar value changed
ItemEvent	ItemListener	itemStateChanged()	Checkbox/Choice/List item selected or deselected
TextEvent	TextListener	textValueChanged()	Text changed in TextField/TextArea
KeyEvent	KeyListener	keyPressed(), keyReleased(), keyTyped()	Keyboard key action
MouseEvent	MouseListener	mouseClicked(), mousePressed(), mouseReleased(), mouseEntered(), mouseExited()	Mouse actions
MouseEvent	MouseMotionListener	mouseMoved(), mouseDragged()	Mouse movement
MouseWheelEvent	MouseWheelListener	mouseWheelMoved()	Mouse wheel rotated
WindowEvent	WindowListener	windowOpened(), windowClosing(), windowClosed(), etc.	Window opened/closed/activated
FocusEvent	FocusListener	focusGained(), focusLost()	Component gets/loses keyboard focus

ComponentEvent	ComponentListener	componentResized(), componentMoved(), componentShown(), componentHidden()	Component resized/moved/shown/hidden
ContainerEvent	ContainerListener	componentAdded(), componentRemoved()	Component added/removed from a container
HierarchyEvent	HierarchyListener	hierarchyChanged()	Component hierarchy changes
HierarchyEvent	HierarchyBoundsListener	ancestorMoved(), ancestorResized()	Parent/ancestor moved or resized

```
public interface ActionListener {

    void actionPerformed(ActionEvent e);

}
```

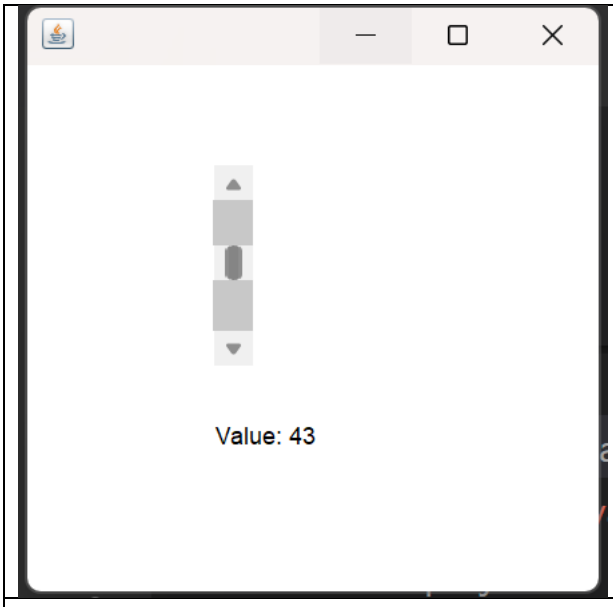
ActionListener

```
import java.awt.*;
import java.awt.event.*;
public class ActionExample extends Frame implements ActionListener {
    Button b;
    Label l;
    ActionExample() {
        b = new Button("Click");
        b.setBounds(100, 80, 80, 30);
        l = new Label();
        l.setBounds(100, 130, 150, 30);
        b.addActionListener(this);
        add(b);
        add(l);
        setSize(300, 250);
        setLayout(null);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {
        l.setText("Button Clicked");
    }
    public static void main(String[] args) {
        new ActionExample();
    }
}
```



Adjustment Listener

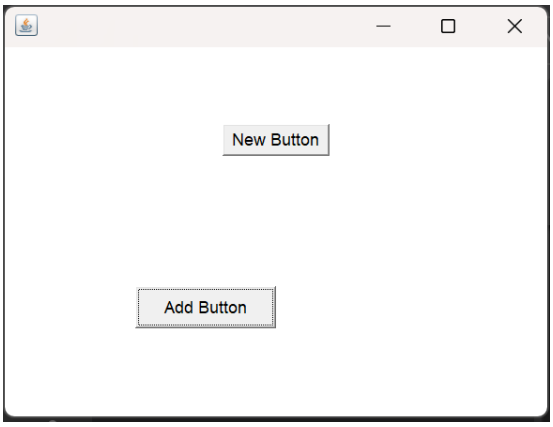
```
import java.awt.*;
import java.awt.event.*;

public class AdjustmentExample extends Frame implements AdjustmentListener {

    Scrollbar s;
    Label l;

    AdjustmentExample() {

        s = new Scrollbar();
        s.setBounds(100, 80, 20, 100);

        l = new Label("Value: 0");
        l.setBounds(100, 200, 150, 30);

        s.addAdjustmentListener(this);

        add(s);
        add(l);

        setSize(300, 300);
        setLayout(null);
        setVisible(true);
    }

    public void adjustmentValueChanged(AdjustmentEvent e) {
        l.setText("Value: " + s.getValue());
    }

    public static void main(String[] args) {
        new AdjustmentExample();
    }
}
```

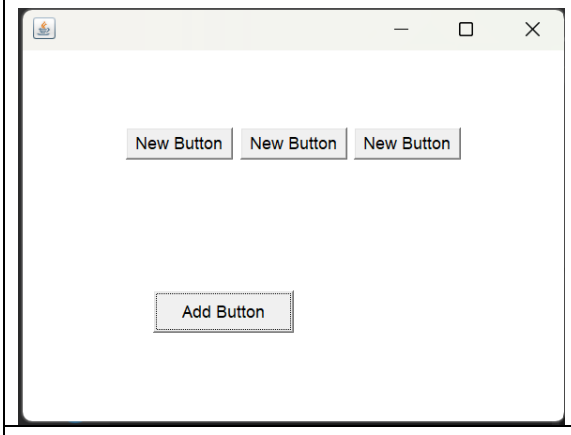


Container Listener

```
import java.awt.*;
import java.awt.event.*;

public class ContainerExample extends Frame implements ContainerListener, ActionListener {

   Button addButton;
   Panel panel;

   ContainerExample() {

      panel = new Panel();
      panel.setBounds(50, 80, 300, 100);

      addButton = new Button("Add Button");
      addButton.setBounds(100, 200, 100, 30);

      add(panel);
      add(addButton);

      // Listen for components added/removed from Frame
      addContainerListener(this);

      // Listen for button click
      addButton.addActionListener(this);

      setSize(400, 300);
      setLayout(null);
      setVisible(true);
   }

   // Called when a component is added
```

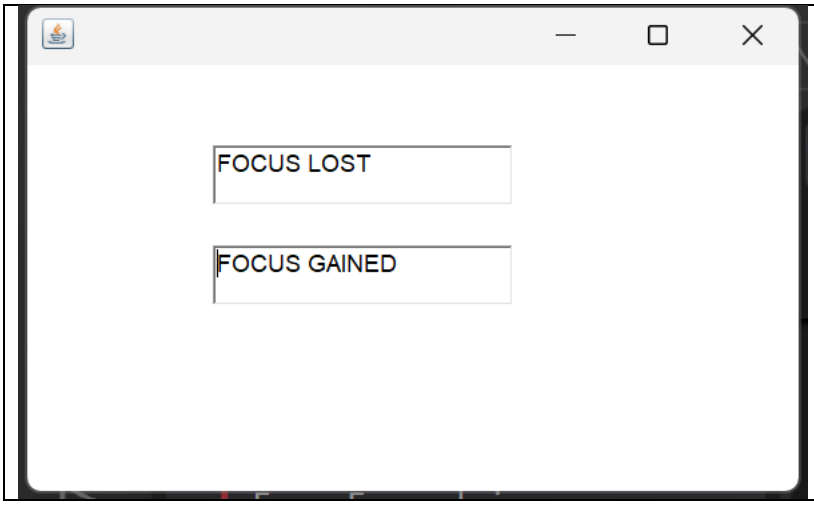
```
    public void componentAdded(ContainerEvent e) {
        //System.out.println("Component Added");
    }

    // Called when a component is removed
    public void componentRemoved(ContainerEvent e) {
        //System.out.println("Component Removed");
    }

    // Called when Add Button is clicked
    public void actionPerformed(ActionEvent e) {

        panel.add(new Button("New Button"));

        panel.validate();
        // recalculate the layout of a container after its contents change
    }

    public static void main(String[] args) {
        new ContainerExample();
    }
}
```





Focus Listener

```
import java.awt.*;
import java.awt.event.*;

public class FocusExample extends Frame implements FocusListener {

    TextField t1, t2;

    FocusExample() {

        t1 = new TextField("First");
        t1.setBounds(100, 70, 150, 30);

        t2 = new TextField("Second");
        t2.setBounds(100, 120, 150, 30);

        add(t1);
        add(t2);

        t1.addFocusListener(this);
        t2.addFocusListener(this);

        setSize(400, 250);
        setLayout(null);
        setVisible(true);
    }

    public void focusGained(FocusEvent e) {
        ((TextField)e.getSource()).setText("FOCUS GAINED");
    }

// e.getSource(): returns the component(obj) that generated the event.

// The component that caused this event is a TextField(obj to TextField)

    public void focusLost(FocusEvent e) {
        ((TextField)e.getSource()).setText("FOCUS LOST");
    }

    public static void main(String[] args) {
        new FocusExample();
    }
}
```

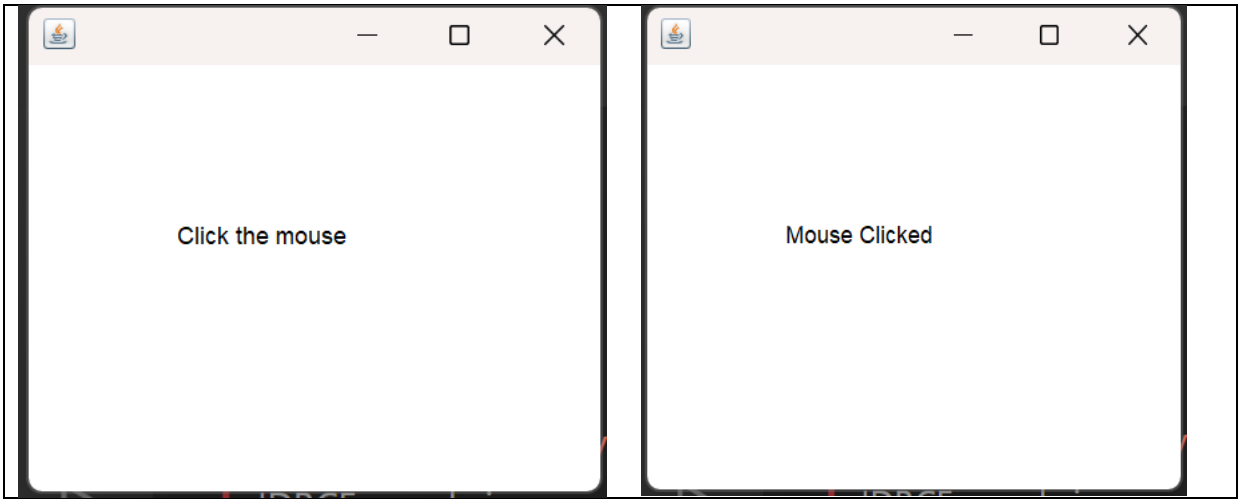

FOCUS LOST
FOCUS GAINED

MouseListener

Method	When it occurs
`mouseClicked()`	Mouse is pressed and released as a click
`mousePressed()`	Mouse button is pressed
`mouseReleased()`	Mouse button is released
`mouseEntered()`	Mouse enters the component
`mouseExited()`	Mouse leaves the component

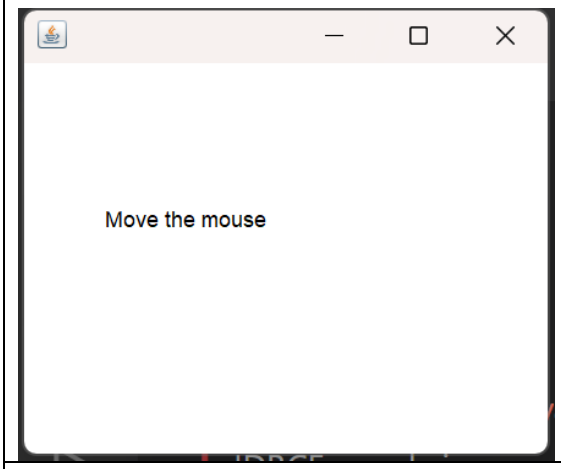
```
import java.awt.*;
import java.awt.event.*;
public class MouseExample extends Frame implements MouseListener {
    Label l;
    MouseExample() {
        l = new Label("Click the mouse");
        l.setBounds(80, 100, 150, 30);
        addMouseListener(this);
        add(l);
        setSize(300, 250);
        setLayout(null);
        setVisible(true);
    }
    public void mouseClicked(MouseEvent e) {
        l.setText("Mouse Clicked");
    }
    public void mousePressed(MouseEvent e) {}
    public void mouseReleased(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}
    public static void main(String[] args) {
        new MouseExample();
    }
}
```

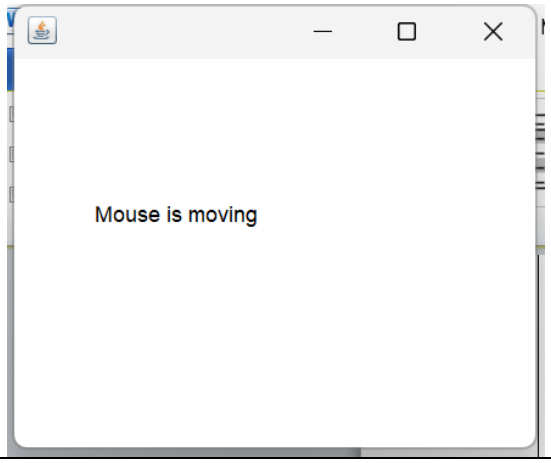
Code	Mouse listener attached to
addMouseListener(this);	Frame
l.addMouseListener(this);	Label


Click the mouse
Mouse Clicked

MouseMotionListener

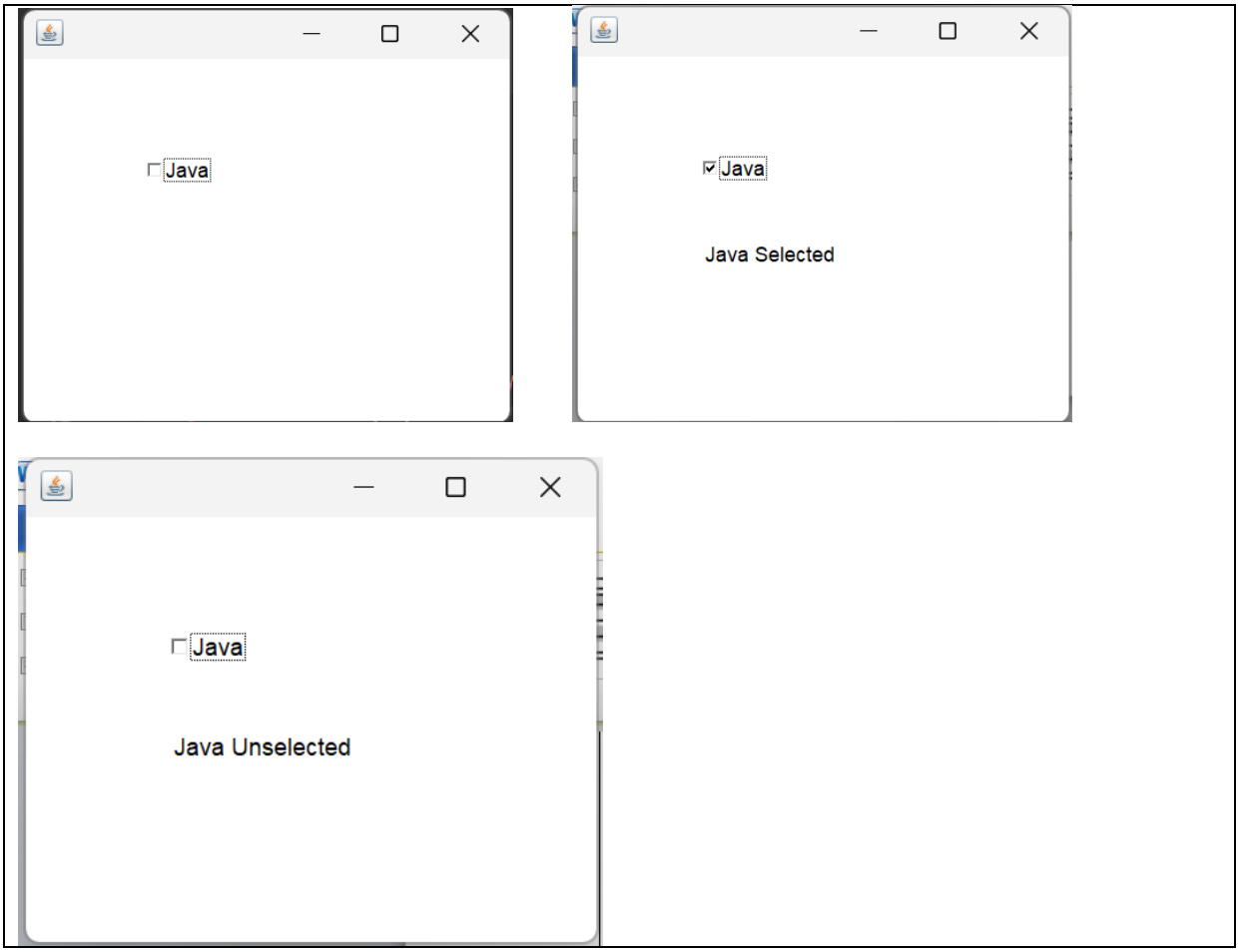
```
import java.awt.*;
import java.awt.event.*;

public class MouseMotionExample extends Frame
        implements MouseMotionListener {

    Label l;

    MouseMotionExample() {

        l = new Label("Move the mouse");
        l.setBounds(50, 100, 200, 30);

        addMouseMotionListener(this);
        add(l);

        setSize(300, 250);
        setLayout(null);
        setVisible(true);
    }

    public void mouseMoved(MouseEvent e) {
        l.setText("Mouse is moving");
    }

    public void mouseDragged(MouseEvent e) {
        l.setText("Mouse is dragged");
    }

    public static void main(String[] args) {
        new MouseMotionExample();
    }
}
```





ItemListener

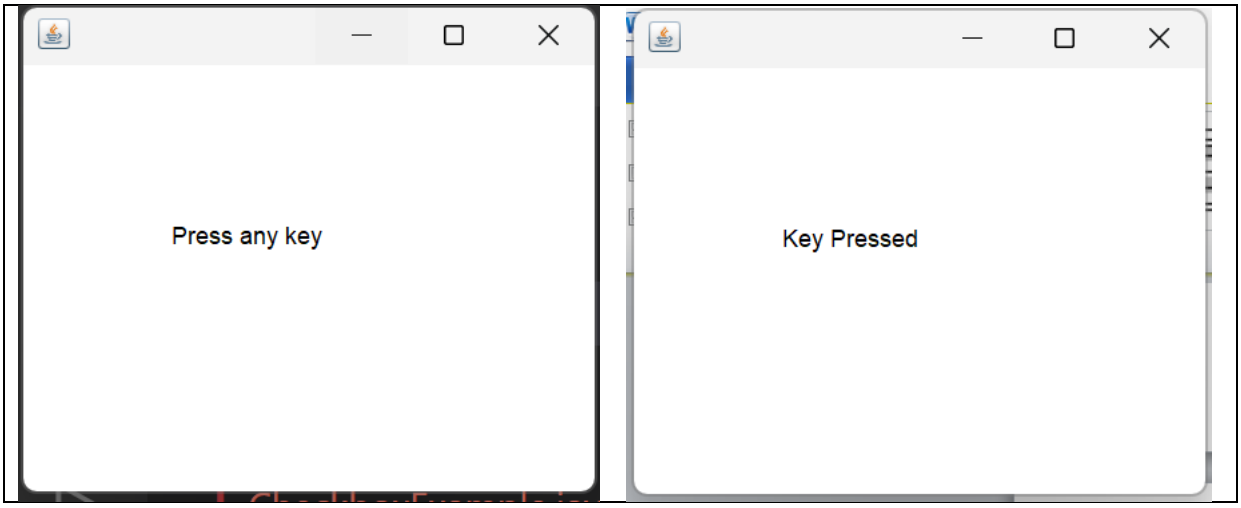
```
import java.awt.*;
import java.awt.event.*;

public class ItemExample extends Frame implements ItemListener {

    Checkbox c;
    Label l;

    ItemExample() {

        c = new Checkbox("Java");
        c.setBounds(80, 80, 100, 30);

        l = new Label();
        l.setBounds(80, 130, 150, 30);

        c.addItemListener(this);

        add(c);
        add(l);

        setSize(300, 250);
        setLayout(null);
        setVisible(true);
    }

    public void itemStateChanged(ItemEvent e) {

        if (c.getState())
            l.setText("Java Selected");
        else
            l.setText("Java Unselected");
    }

    public static void main(String[] args) {
        new ItemExample();
    }
}
```


Java
Java
Java Selected
Java
Java Unselected

KeyListener

Method	When it occurs	Example
keyPressed(KeyEvent e)	When a key is pressed down	Press A
keyReleased(KeyEvent e)	When a key is released	Release A
keyTyped(KeyEvent e)	When a character is typed	Typing A or 5

```
import java.awt.*;
import java.awt.event.*;

public class KeyExample extends Frame implements KeyListener {

   Label l;

   KeyExample() {

      l = new Label("Press any key");
      l.setBounds(80, 100, 150, 30);

      addKeyListener(this);
      add(l);

      setSize(300, 250);
      setLayout(null);
      setVisible(true);

      requestFocus();
      // Request keyboard focus for the Frame.
   }

   public void keyPressed(KeyEvent e) {
      l.setText("Key Pressed");
   }

   public void keyReleased(KeyEvent e) {
   }

   public void keyTyped(KeyEvent e) {
   }

   public static void main(String[] args) {
      new KeyExample();
   }
}
```


Press any key
Key Pressed

WindowListener

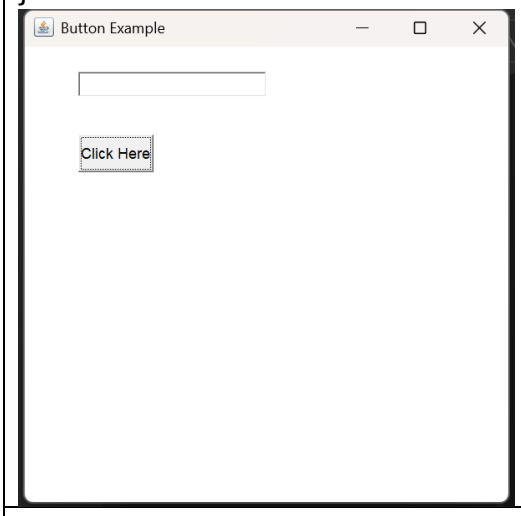
Method	When it is called	Example
windowOpened(WindowEvent e)	When the window is **opened**	Frame becomes visible
windowClosing(WindowEvent e)	When the user **tries to close** the window	Click **X** button
windowClosed(WindowEvent e)	After the window is **actually closed**	Window has been disposed
windowIconified(WindowEvent e)	When the window is **minimized**	Click minimize
windowDeiconified(WindowEvent e)	When a minimized window is **restored**	Restore the window
windowActivated(WindowEvent e)	When the window becomes **active**	Select/click the window
windowDeactivated(WindowEvent e)	When the window becomes **inactive**	Select another window

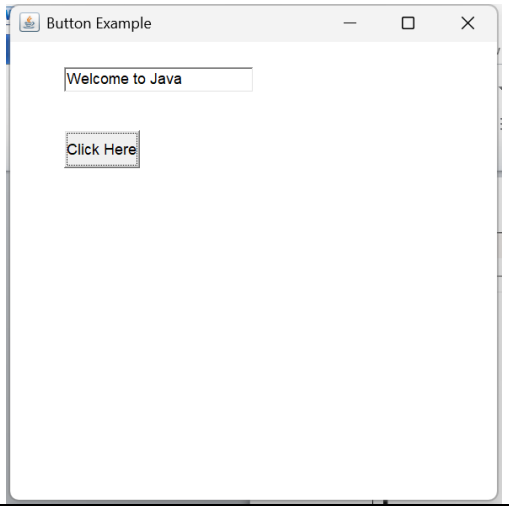
```
import java.awt.*;
import java.awt.event.*;

public class WindowExample extends Frame
        implements WindowListener {

    WindowExample() {

        addWindowListener(this);

        setSize(300, 250);
        setVisible(true);
    }

    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }

    public void windowOpened(WindowEvent e) {}
    public void windowClosed(WindowEvent e) {}
    public void windowIconified(WindowEvent e) {}
    public void windowDeiconified(WindowEvent e) {}
    public void windowActivated(WindowEvent e) {}
    public void windowDeactivated(WindowEvent e) {}

    public static void main(String[] args) {
        new WindowExample();
    }
}
```

Button Events Method 1

```
import java.awt.*;
import java.awt.event.*;

public class ButtonExample3 {

public static void main(String[] args) {

Frame f = new Frame("Button Example");

    TextField tf=new TextField();
    tf.setBounds(50,50, 150,20);
```

```
    Button b=new Button("Click Here");
    b.setBounds(50,100,60,30);

    b.addActionListener(new ActionListener() {

    public void actionPerformed (ActionEvent e) {
            tf.setText("Welcome to Java");
        }
    });
    // Anonymous class
    f.add(b);
    f.add(tf);
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
}
}
```





Additional Information

new ActionListener(); // No direct interface object

Java syntax for an anonymous class is
new InterfaceName() {
  // implementation of interface methods
}

new ActionListener() {
  public void actionPerformed(ActionEvent e) {
  }
}; // anonymous class object

The same thing could be written as:(using class name**)**
class MyListener implements ActionListener {
 	
  public void actionPerformed(ActionEvent e) {
  tf.setText("Welcome to Java");
  }

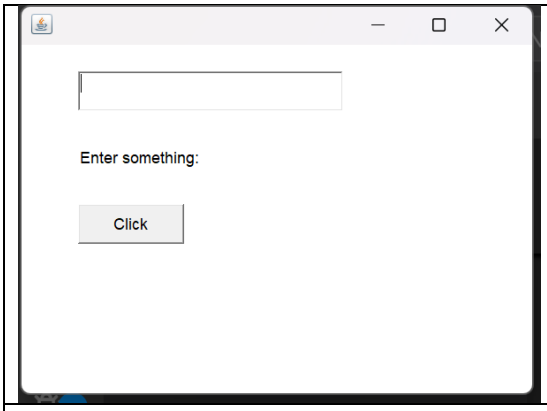
```
}


MyListener obj = new MyListener();
b.addActionListener(obj);
```

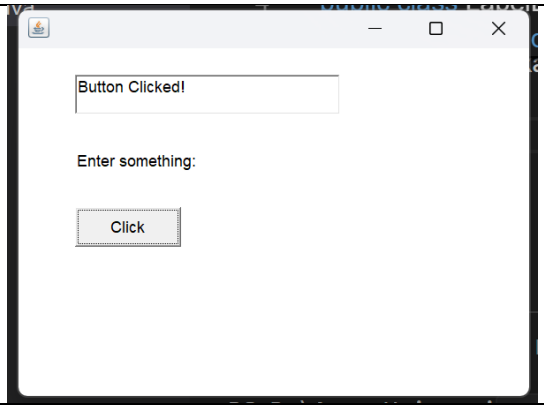
Equivalen Lambda Expression

```
b.addActionListener(e -> {tf.setText("Welcome to Java");});
```

Button Events Method 2

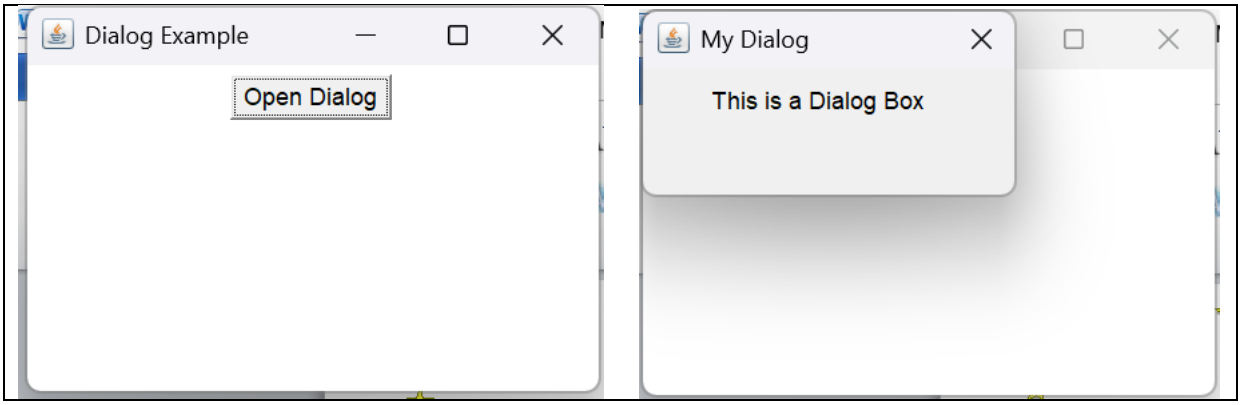
```
import java.awt.*;
import java.awt.event.*;

public class LabelExample2 extends Frame implements ActionListener {
    TextField tf;
    Label l;
    Button b;

    LabelExample2() {
        tf = new TextField();
        tf.setBounds(50, 50, 200, 30);
        l = new Label("Enter something:");
        l.setBounds(50, 100, 150, 30);
        b = new Button("Click");
        b.setBounds(50, 150, 80, 30);
        b.addActionListener(this);
        add(tf);
        add(l);
        add(b);
        setSize(400, 300);
        setLayout(null);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {
        tf.setText("Button Clicked!");
    }
    public static void main(String[] args) {
        new LabelExample2();
    }
}
```





AWT Dialog Box

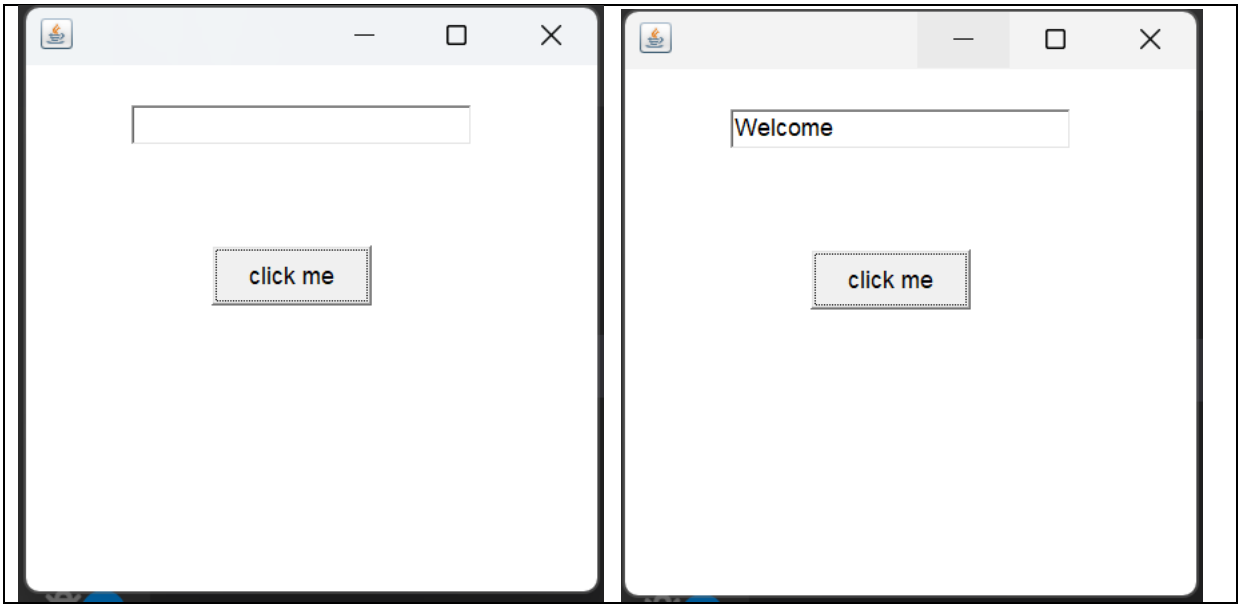
```
import java.awt.*;
import java.awt.event.*;

public class DialogExample {
    public static void main(String[] args) {

        Frame f = new Frame("Dialog Example");

        Button b = new Button("Open Dialog");

        f.add(b);

        f.setSize(300, 200);
        f.setLayout(new FlowLayout());
        f.setVisible(true);

        // Open dialog when button is clicked
        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {

                Dialog d = new Dialog(f, "My Dialog");

                d.setSize(200, 100);
                d.setLayout(new FlowLayout());

                d.add(new Label("This is a Dialog Box"));

                d.setVisible(true);
            }
        });
    }
}
```



APPROACHES FOR EVENT HANDLING

Event handling within the class

```
import java.awt.*;
import java.awt.event.*;
class AEvent extends Frame implements ActionListener{
TextField tf;
AEvent(){

//create components
tf=new TextField();
tf.setBounds(60,50,170,20);
Button b=new Button("click me");
b.setBounds(100,120,80,30);

//register listener
b.addActionListener(this);//passing current instance

//add components and set size, layout and visibility
add(b);add(tf);
setSize(300,300);
setLayout(null);
setVisible(true);
}
public void actionPerformed(ActionEvent e){
tf.setText("Welcome");
}
public static void main(String args[]){
new AEvent();
}
}
```



Event Handling by Outer/Another Class

```
import java.awt.*;
import java.awt.event.*;
class AEvent2 extends Frame{
TextField tf;
AEvent2(){

//create components
tf=new TextField();
tf.setBounds(60,50,170,20);
Button b=new Button("click me");
b.setBounds(100,120,80,30);

//register listener
Outer o=new Outer(this);
//current AEvent2 object, which is given to Outer so that Outer can access AEvent2's components like tf

b.addActionListener(o);    //passing outer class instance

//add components and set size, layout and visibility
add(b);
add(tf);
setSize(300,300);
setLayout(null);
setVisible(true);
}

public static void main(String args[]){
new AEvent2();
```

```
}
}
```

```
//Outer handles the button event
import java.awt.event.*;
class Outer implements ActionListener{
AEvent2 obj;
Outer(AEvent2 obj){
this.obj=obj;
}
public void actionPerformed(ActionEvent e){
obj.tf.setText("welcome");
}
}
```

Event Handling by Anonymous Class

```
import java.awt.*;
import java.awt.event.*;

class AEvent3 extends Frame{
TextField tf;
AEvent3(){
tf=new TextField();
tf.setBounds(60,50,170,20);
Button b=new Button("click me");
b.setBounds(50,120,80,30);

b.addActionListener(new ActionListener(){
public void actionPerformed(){
tf.setText("hello");
}
});
add(b);add(tf);
setSize(300,300);
setLayout(null);
setVisible(true);
}
public static void main(String args[]){
new AEvent3();
}
}
```